

APOSTILA DO ALUNO — AULA 4

UML — Diagrama de Casos de Uso

Curso: Técnico em Desenvolvimento de Sistemas | Turma: 103513 | Disciplina: Modelagem de Sistemas (82h + 04h planejamento)

Data: 18/09/2026 (sexta-feira) | Horário: 07:30 às 11:30 (4h)

Objetivos desta aula

- Revisar e consolidar os principais conceitos das Aulas 1 a 3 (regra de negócio, requisitos funcionais e não funcionais, técnicas de análise, MoSCoW, critério de aceite e visão geral de UML).
- Reconhecer os elementos do Diagrama de Casos de Uso.
- Aplicar os relacionamentos associação, include, extend e generalização.
- Construir o Diagrama de Casos de Uso do sistema TechBurger.

Revisão Geral: Aulas 1 a 3

Antes de entrar no assunto de hoje, um raio-X rápido de tudo que já vimos — da regra de negócio até a visão geral da UML. Use esta seção para reativar a memória; se precisar de mais detalhe, volte à apostila da aula correspondente.

Aula 1 — Regra de Negócio

Uma regra de negócio é uma diretriz que define ou restringe algum aspecto do negócio, independente de haver ou não um sistema informatizado. Toda regra pode ser decomposta em Condição (o “se”), Ação (o “então”) e Exceção (quando a regra deixa de valer).

Tipo de regra	O que faz	Exemplo TechBurger
Restrição (constraint)	Limita ou bloqueia uma ação sob condição.	“Pedidos abaixo de R\$ 15,00 não podem ser confirmados pelo aplicativo.”
Fato / definição	Define um termo ou relacionamento, sem ação.	“Um pedido pertence a um único cliente.”
Cálculo (derivação)	Deriva um novo valor a partir de outros já existentes.	“O total do pedido é a soma dos itens menos os descontos.”
Disparador de ação (action enabler)	Aciona automaticamente um evento quando a condição ocorre.	“Ao atingir 100 pontos, o combo grátis é concedido automaticamente.”

Boas práticas: uma regra de negócio bem escrita é atômica (uma única decisão), verificável (dá para checar objetivamente), redigida na linguagem do negócio (sem termos técnicos) e rastreável a um dono.

A cadeia de requisitos liga o negócio ao sistema em três níveis: requisito de negócio (por que o sistema precisa existir), requisito de usuário (o que a pessoa precisa conseguir fazer) e requisito de sistema/funcional (o que exatamente o sistema deve fazer).

Aula 2 — Requisito Funcional e Não-Funcional

Um Requisito Funcional (RF) descreve o que o sistema deve fazer, redigido no padrão “o sistema deve + verbo de ação”, sempre objetivo e verificável. Um Requisito Não-Funcional (RNF) descreve como o sistema se comporta — não é uma funcionalidade, e sim uma qualidade do sistema.

Categoria de RNF	Exemplo TechBurger
Desempenho	O sistema deve confirmar o pagamento em até 3 segundos após o envio.
Segurança	Os dados de cartão e PIX devem ser criptografados em trânsito e em repouso.
Usabilidade	O aplicativo deve permitir finalizar um pedido simples em no máximo 4 toques.
Disponibilidade	O sistema deve estar disponível 99,5% do tempo durante o funcionamento da loja.
Confiabilidade	O sistema não deve permitir a perda de um pedido já pago, mesmo com queda de conexão.
Manutenibilidade	Um desenvolvedor novo deve corrigir um bug simples no módulo de pedidos em até 1 dia.
Portabilidade	O aplicativo deve funcionar em Android e iOS sem alterar as regras de negócio.
Escalabilidade	O sistema deve suportar 3x o volume atual de pedidos sem queda de desempenho.

Teste rápido para não confundir os dois: se a frase descreve uma ação que o sistema executa, é RF; se descreve uma qualidade de como essa ação acontece, é RNF.

Técnica de análise	Quando usar
Entrevista	Poucos stakeholders, tema complexo e sensível.
Questionário	Centenas de usuários finais espalhados.
Observação	O usuário não sabe explicar o que faz, só faz.
Brainstorming	Início de projeto, ainda sem direção clara.
Prototipação	Validar uma tela ou fluxo antes de programar de verdade.
Análise de documentos	Regras já existem, mas não estão documentadas.
Workshop de requisitos (JAD)	Muitos decisores, prazo curto para alinhar tudo.
Categoria MoSCoW	Exemplo TechBurger
Must have	“Enviar o pedido à cozinha somente após confirmação do pagamento.” — essencial.
Should have	“Aplicar 5% de desconto automático quando o pagamento for via PIX.” — importante, mas não trava o sistema.
Could have	“Sugerir um item substituto quando o item estiver em falta.” — desejável.
Won't have (por agora)	“Recomendar produtos usando inteligência artificial.” — fora do escopo atual.

Um critério de aceite é uma condição objetiva que diz quando um RF pode ser considerado “pronto e correto”, no formato Dado [contexto inicial] / Quando [uma ação acontece] / Então [o sistema deve reagir assim] — esse formato é conhecido como Gherkin. Exemplo: para o RF “o sistema deve bloquear o envio do pedido até que o pagamento seja confirmado” — Dado um pedido com pagamento pendente, Quando o atendente tentar enviar o pedido à cozinha, Então o sistema deve impedir o envio e exibir uma mensagem de pagamento pendente.

Aula 3 — Modelagem de Sistemas e UML

Modelagem de sistemas é o processo de representar, por meio de modelos, o funcionamento de um sistema antes ou durante o desenvolvimento — corrigir um erro no modelo é muito mais barato do que corrigir código já escrito.

Fase em que o erro é encontrado	Custo relativo de corrigir
Levantamento de requisitos	1x
Modelagem / projeto	5x
Codificação	10x
Produção (sistema já no ar)	100x ou mais

Modelos estruturais representam o que existe (classes, objetos, componentes); modelos comportamentais representam o que acontece (casos de uso, atividades, sequência). No TechBurger: como estrutura, o pedido envolve um Pedido, vários Itens e um Cliente; como comportamento, é a sequência cliente escolhe os itens → confirma pagamento → pedido vai para a cozinha.

Diagrama (o que a disciplina aprofunda)	Quando estudamos
Casos de Uso (comportamental) — quem interage e o que pode fazer	Aula 4 (hoje!)
Classes (estrutural) — entidades, atributos e relações	Aulas 5 e 6
Atividades (comportamental) — passo a passo de um processo	Aula 7
Sequência (comportamental) — troca de mensagens entre objetos	Aula 8

Panorama de reconhecimento (não são foco da disciplina, mas aparecem no mercado): Diagrama de Estados (estados de um objeto e o que dispara a mudança), Componentes (como o sistema se divide em partes que se comunicam), Objetos (uma instância específica, com valores reais) e Implantação (como as partes são distribuídas em servidores/nuvem).

Ferramenta usada nesta disciplina: draw.io / diagrams.net (gratuita, no navegador). Outras do mercado: StarUML/Astah (validação de notação), Lucidchart (colaborativa em nuvem), PlantUML (diagrama a partir de texto, versionável com o código).

Anti-padrões de modelagem a evitar: modelar demais (ninguém lê), modelar de menos (não ajuda a tirar dúvida), diagrama desatualizado (pior que não ter diagrama), notação inventada, modelar sem validar com quem usa, e copiar um modelo pronto da internet sem adaptar às regras do seu sistema.

Quiz relâmpago geral (revisão)

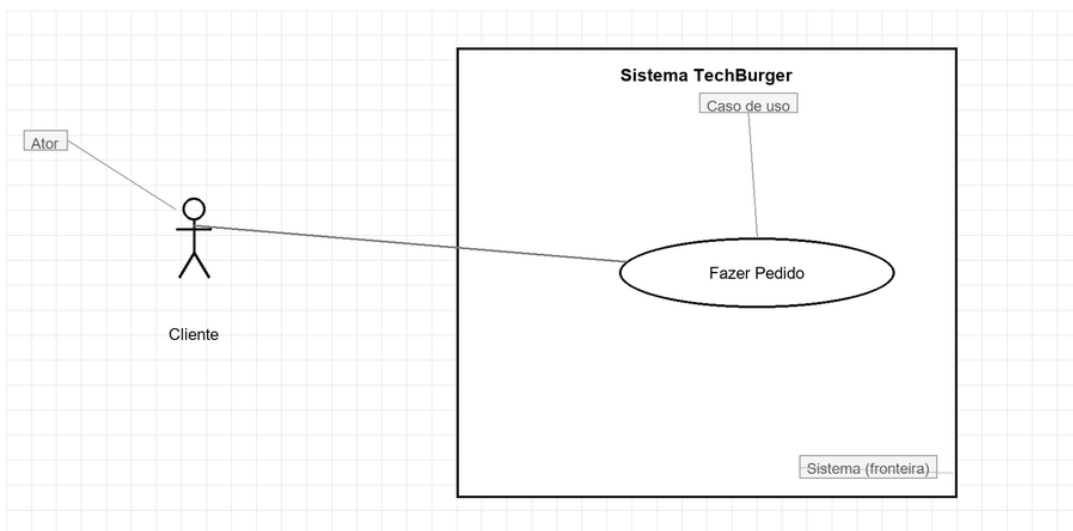
Testem o que lembram das Aulas 1 a 3. Respostas no Gabarito.

1. Classifique: “Um pedido pertence a um único cliente.” — é Restrição, Fato/definição, Cálculo ou Disparador de ação?
2. “O sistema deve emitir a nota fiscal em até 5 segundos.” Isso é um RF ou um RNF? De qual categoria?
3. Vocês precisam validar uma tela antes de programar de verdade. Qual técnica de análise de requisitos é mais indicada?
4. No MoSCoW, um requisito “importante, mas o sistema ainda funciona sem ele no lançamento” entra em qual categoria?
5. Escreva o início de um critério de aceite no formato Dado/Quando/Então para o RF “o sistema deve liberar a entrega apenas até 5 km de distância”.
6. Um diagrama mostra bonecos fora de um retângulo, e elipses dentro. Isso é um modelo estrutural ou comportamental? Qual diagrama é esse?

Fechando a revisão: regra de negócio → RF/RNF → critério de aceite → modelo em UML. Hoje continuamos essa cadeia com o primeiro diagrama de verdade: Casos de Uso, com o mesmo cenário TechBurger de sempre.

5. Elementos do Diagrama de Casos de Uso

Elemento	O que representa	Notação
Ator	Quem ou o que interage com o sistema (pessoa, outro sistema)	Boneco palito com o nome embaixo
Caso de uso	Uma funcionalidade que o sistema oferece	Elipse com o nome dentro (verbo + complemento)
Sistema (fronteira)	Delimita o que está dentro/fora do sistema	Retângulo envolvendo os casos de uso



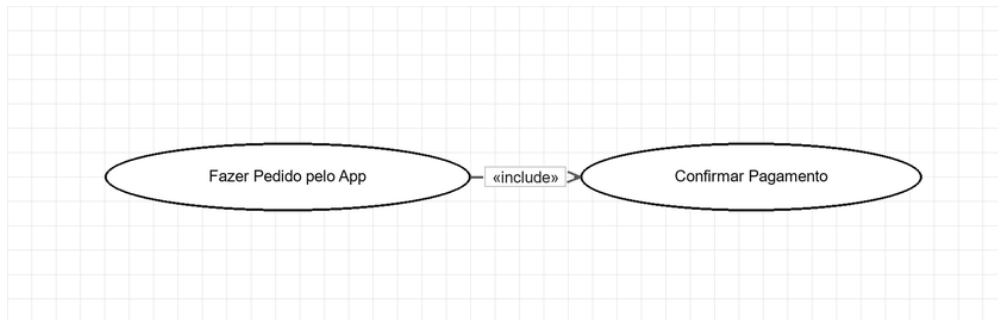
Notação: Ator, Caso de uso e Sistema (fronteira).

6. Relacionamentos

- Associação — linha simples entre ator e caso de uso.
- «include» — um caso de uso sempre inclui outro como parte obrigatória do fluxo.
- «extend» — um caso de uso estende outro de forma opcional/condicional.

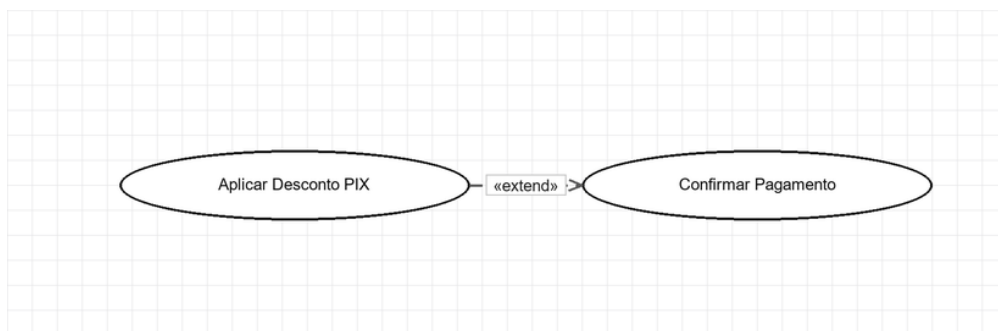
- Generalização entre atores — um ator é um tipo mais específico de outro (seta com ponta triangular vazada).

Exemplo do TechBurger: “Fazer Pedido pelo App” inclui (include) “Confirmar Pagamento”, porque todo pedido feito pelo app passa obrigatoriamente por essa etapa — não existe pedido sem confirmar o pagamento.



Exemplo: «include» no TechBurger — Fazer Pedido pelo App inclui Confirmar Pagamento.

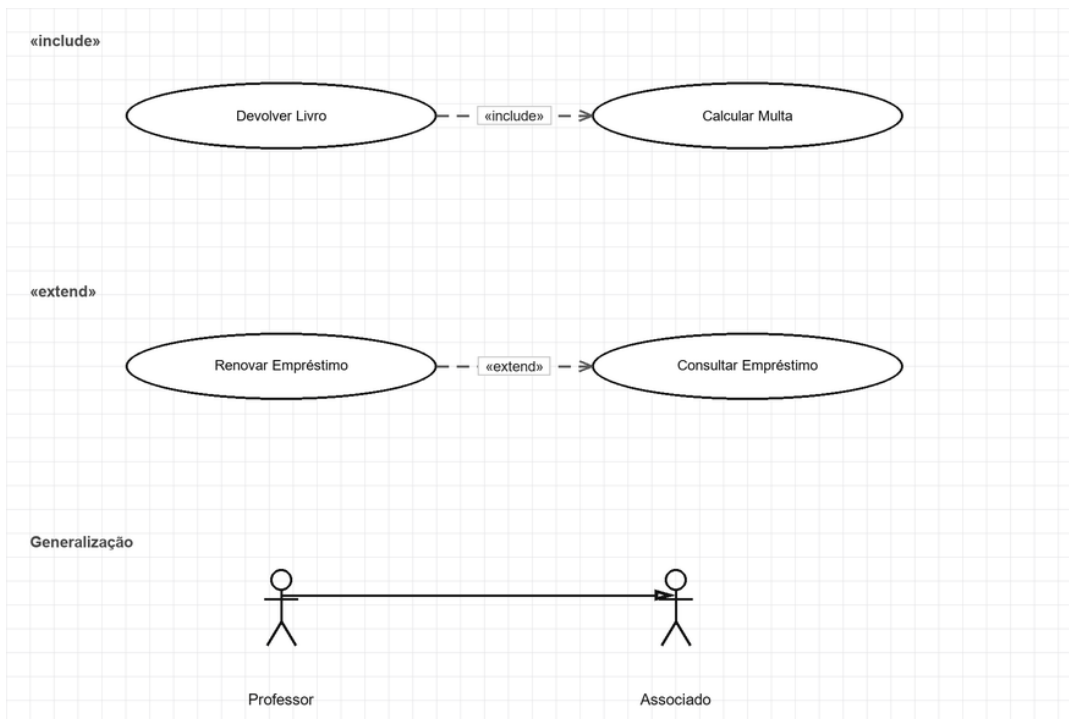
Exemplo do TechBurger: “Aplicar Desconto PIX” estende (extend) “Confirmar Pagamento”, porque só acontece quando o cliente escolhe pagar via PIX.



Exemplo: «extend» no TechBurger — Aplicar Desconto PIX estende Confirmar Pagamento.

Include e extend também aparecem em outros domínios — vejam o mesmo padrão na Biblioteca “Ler Mais”:

Relacionamento	Exemplo — Biblioteca “Ler Mais”
«include»	“Devolver Livro” «include» “Calcular Multa” — toda devolução passa pelo cálculo de multa (mesmo que o resultado seja R\$ 0,00): é um passo obrigatório, não uma exceção.
«extend»	“Renovar Empréstimo” «extend» “Consultar Empréstimo” — só acontece se o associado decidir renovar; nem toda consulta vira renovação.
Generalização	“Professor” é um tipo mais específico de “Associado”: faz tudo que um associado comum faz, mas tem uma regra própria (prazo de empréstimo estendido para 30 dias).



Exemplo: include, extend e generalização na Biblioteca “Ler Mais”.

Regra prática para decidir entre include e extend: pergunte “isso acontece sempre (include) ou só às vezes, sob uma condição (extend)?”.

7. O cenário TechBurger — atores e casos de uso

Vamos modelar o mesmo sistema TechBurger apresentado na Aula 1. Estes são os atores e casos de uso principais identificados a partir das regras de negócio:

Atores:

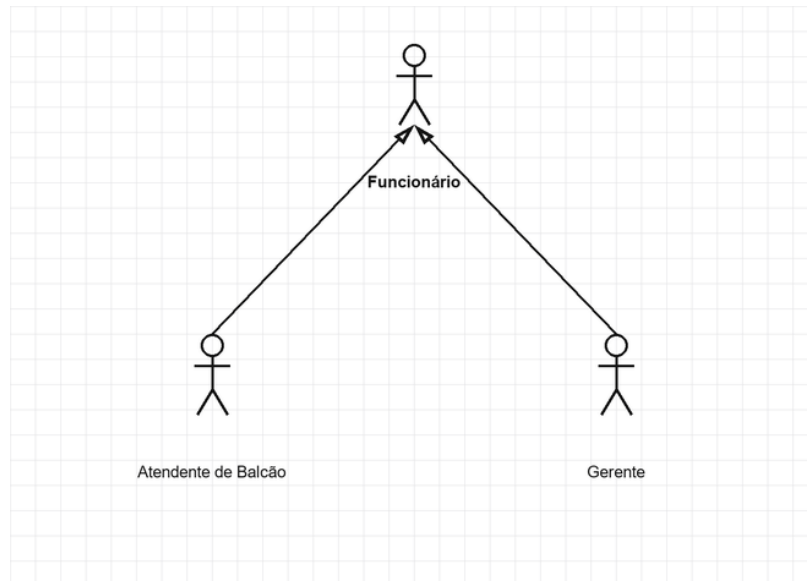
- Cliente
- Atendente de Balcão
- Cozinha
- Gerente
- Sistema de Pagamento (ator externo)

Casos de uso principais:

Caso de uso	Ator / observação
Fazer Pedido pelo App	Cliente
Fazer Pedido no Balcão	Cliente / Atendente de Balcão
Confirmar Pagamento	Sistema de Pagamento
Aplicar Desconto PIX	«extend» de Confirmar Pagamento
Acumular Pontos de Fidelidade	Cliente
Resgatar Combo Grátis	Cliente
Validar Endereço de Entrega	Cliente
Sugerir Item Substituto	«extend» de Fazer Pedido

Autorizar Pedido Grande	Gerente
Recusar Pedido de Cliente Bloqueado	«extend» de Fazer Pedido pelo App

Dica: Atendente de Balcão e Gerente podem ser modelados como tipos mais específicos de um ator mais geral “Funcionário” (generalização).



Exemplo: generalização — Atendente de Balcão e Gerente são tipos de Funcionário.

8. Erros comuns ao modelar Casos de Uso

Erro comum	Correção
Transformar um passo de tela em caso de uso (“Clicar no botão confirmar”)	Um caso de uso descreve um objetivo do ator, não um clique específico. Use “Confirmar Pedido”.
Usar «include»/«extend» sem necessidade real, “para ficar mais completo”	Se o relacionamento não ajuda a entender o fluxo, ele só polui o diagrama.
Colocar um ator dentro da fronteira do sistema	Atores interagem de FORA para DENTRO; dentro do retângulo só ficam casos de uso.
Nomear caso de uso sem verbo no infinitivo (“Pedido”, “Pagamento”)	Use verbo + complemento: “Fazer Pedido”, “Confirmar Pagamento”.
Esquecer atores externos que são outros sistemas	O Sistema de Pagamento é um ator, mesmo sendo outro sistema, não uma pessoa.
Duplicar o mesmo caso de uso com nomes diferentes (“Fazer Pedido” e “Realizar Pedido”)	Padronizem o vocabulário em grupo antes de começar a desenhar.

- Passo de tela como caso de uso — errado: “Clicar no botão confirmar”; certo: “Confirmar Pedido”. O caso de uso descreve um objetivo do ator, não um clique específico na interface.
- Include/extend sem necessidade real — usar «include»/«extend» para tudo só porque “fica mais completo” só polui o diagrama; pergunte sempre se aquilo acontece sempre (include) ou às vezes (extend).
- Ator dentro da fronteira do sistema — atores representam quem interage de fora para dentro; colocá-los dentro do retângulo confunde “quem usa” com “o que o sistema faz”.
- Caso de uso sem verbo no infinitivo — errado: “Pedido”, “Pagamento”; certo: “Fazer Pedido”, “Confirmar Pagamento”. Sem o verbo, não fica claro qual ação o sistema realiza.

- Esquecer atores externos (outros sistemas) — o Sistema de Pagamento é um ator, mesmo sendo outro sistema, não uma pessoa; esquecer atores-sistema deixa o diagrama incompleto.
- Duplicar o mesmo caso de uso com nomes diferentes — “Fazer Pedido” e “Realizar Pedido” não podem aparecer como dois casos de uso distintos; padronizem o vocabulário em grupo antes de desenhar.

9. Atividade prática

- Em grupo, desenhem (no draw.io ou papel) o Diagrama de Casos de Uso completo do TechBurger.
- Devem constar: os 5 atores, pelo menos 8 dos 10 casos de uso listados, pelo menos 1 relacionamento «include» ou «extend», e a generalização entre Atendente/Gerente e Funcionário.
- Tempo: 60 minutos. Ao final, um representante do grupo apresenta o diagrama para a turma.

Glossário

Termo	Significado
Ator	Papel externo que interage com o sistema.
Caso de uso	Funcionalidade oferecida pelo sistema a um ator.
Include	Relacionamento de inclusão obrigatória entre casos de uso.
Extend	Relacionamento de extensão opcional/condicional entre casos de uso.
Generalização	Relação de “é um tipo de” entre atores (ou entre casos de uso).

- Ator — papel externo que interage com o sistema.
- Caso de uso — funcionalidade oferecida pelo sistema a um ator.
- Sistema (fronteira) — retângulo que delimita o que está dentro do sistema do que está fora.
- Associação — linha simples que liga um ator a um caso de uso que ele realiza.
- Include — relacionamento de inclusão obrigatória entre casos de uso.
- Extend — relacionamento de extensão opcional/condicional entre casos de uso.
- Generalização — relação de “é um tipo de” entre atores (ou entre casos de uso).

Perguntas de revisão

1. Qual a diferença entre ator e caso de uso?
2. Quando usar «include» e quando usar «extend»?
3. Dê um exemplo de generalização entre atores no cenário TechBurger.
4. Por que “clique no botão confirmar” não é um bom nome de caso de uso?
5. A que caso de uso o ator Sistema de Pagamento se conecta diretamente? E qual outro caso de uso se relaciona a ele só indiretamente, via «extend»?
6. No exemplo da Biblioteca “Ler Mais”, por que “Devolver Livro” inclui “Calcular Multa” por «include» e não por «extend»?
7. Por que “Renovar Empréstimo” é um «extend» de “Consultar Empréstimo” na Biblioteca “Ler Mais”, e não um «include»?
8. O que a generalização entre “Professor” e “Associado” representa na Biblioteca “Ler Mais”? O que o Professor herda, e o que tem de próprio?